

e just estimates to visualize the plan better, these are not deadlines and can change at any time.

mermaid

gantt

title CI Data Partitioning Timeline

dateFormat YYYY-MM-DD

axisFormat %m-%y

section Phase 0

Build data partitioning strategy :done, 01, 2022-06-01, 90d

section Phase 1

Partition biggest CI tables :11, after 01, 200d

Biggest table partitioned :milestone, metadata, 2023-03-01, 1min

Tables larger than 100GB partitioned :milestone, 100gb, after 11, 1min

section Phase 2

Add partitioning keys to SQL queries :21, 2023-01-01, 120d

Emergency partition detachment possible :milestone, detachment, 2023-04-01, 1min

All SQL queries are routed to partitions :milestone, routing, after 21, 1min

section Phase 3

Build new data access patterns :31, 2023-05-01, 120d

New API endpoint created for inactive data :milestone, api1, 2023-07-01, 1min

Filtering added to existing API endpoints :milestone, api2, 2023-09-01, 1min

section Phase 4

Introduce time-decay mechanisms :41, 2023-08-01, 120d

Inactive partitions are not being read :milestone, part1, 2023-10-01, 1min

Performance of the database cluster improves :milestone, part2, 2023-11-01, 1min

section Phase 5

Introduce auto-partitioning mechanisms :51, 2023-09-01, 120d

New partitions are being created automatically :milestone, part3, 2023-12-01, 1min

Partitioning is made available on self-managed :milestone, part4, 2024-01-01, 1min

Conclusions

We want to build a solid strategy for partitioning CI/CD data. We are aware of the fact that it is difficult to iterate on this design, because a mistake made in managing the database schema of our multi-terabyte PostgreSQL instance might not be easily reversible without potential downtime. That is the reason we are spending a significant amount of time to research and refine our partitioning strategy. The strategy, described in this document, is subject to iteration as well. Whenever we find a better way to reduce the risk and improve our plan, we should update this document as well.

We've managed to find a way to avoid large-scale data migrations, and we are building an iterative strategy for partitioning CI/CD data. We documented our strategy here to share knowledge and solicit feedback from other team members.

Who

DRIs:

<!-- vale gitlab.Spelling = NO -->

Role	Who	
----- -----		
Author	Grzegorz Bizon, Principal Engineer	
Recommender	Kamil Trzeciński, Senior Distinguished Engineer	
Product Leadership	Jackie Porter, Director of Product Management	
Engineering Leadership	Caroline Simpson, Engineering Manager / Cheryl Li, Senior Engineering Manager	
Lead Engineer	Marius Bobin, Senior Backend Engineer	
Senior Engineer	Maxime Orefice, Senior Backend Engineer	
Senior Engineer	Tianwen Chen, Senior Backend Engineer	

<!-- vale gitlab.Spelling = YES -->

SECTION: engineering/architecture/design-documents/ci_data_decay/reduce_data_growth_rate.md

{{< engineering/design-document-header >}}

Problem to solve

TODO

Strategies

Delete pipeline processing data

Once a build gets archived, it is no longer possible to resume pipeline processing in such pipeline. It means that all the metadata, we store in PostgreSQL, that is needed to efficiently and reliably process builds can be safely moved to a different data store.

Storing pipeline processing data is expensive as this kind of CI/CD data represents a significant portion of data stored in CI/CD tables. Once we restrict access to processing archived pipelines, we can move this metadata to a different place - preferably object storage - and make it accessible on demand, when it is really needed again (for example for compliance or auditing purposes).

We need to evaluate whether moving data is the most optimal solution. We might be able to use de-duplication of metadata entries and other normalization strategies to consume less storage while retaining ability to query this dataset. Technical evaluation will be required to find the best solution here.

Epic: Reduce the rate of builds metadata table growth.

SECTION: engineering/architecture/design-documents/ci_data_decay/retention_policies.md

{{< engineering/design-document-header >}}

Problem to solve

TODO

Strategies

SECTION: engineering/architecture/design-documents/ci_gcp_secrets_manager/_index.md

{{< engineering/design-document-header >}}

Summary

This blueprint describes the architecture to add GCP Secrets Manager as one of the sources for CI External Secrets.

Motivation

GitLab CI allows users to pull secrets from external sources into GitLab CI jobs. Prior to this, the supported secret managers are HashiCorp Vault and Azure Key Vault. GCP Secrets Manager is another major secret manager product and there has been multiple requests and feedback to add GCP Secrets Manager to the list of supported secret managers.

Goals

The goal of this feature is to allow GitLab CI users to use secrets stored in GCP Secrets Manager in their CI jobs.

Non-Goals

This feature does not cover the following:

- Using secrets from GCP Secrets Manager in other GitLab workloads.
- Managing secrets in GCP Secrets Manager or other secret managers through GitLab.

Proposal

This feature requires a tight integration between GCP Secrets Manager, GitLab Rails and GitLab Runner.

The solution to this feature involves three main parts:

1. Authentication with GCP Secrets Manager

1. CI configuration on GitLab Rails
1. Secrets access by GitLab Runner

Authentication with GCP Secrets Manager

GCP Secrets Manager needs to authenticate secret access requests coming from GitLab Runner. Since GitLab Runner can operate in many modes (GitLab.com SaaS runners, SaaS with self-managed runner, GitLab Self-Managed, etc), there is no direct correlation between the Runner instance and any GCP identities that can have access to the secrets.

To solve this, we would use OIDC and GCP's Workload Identity Federation mechanism to authorize the requests.

CI jobs already have support for OIDC through CI variables containing ID tokens issued by the GitLab instance.

These ID tokens already carry claims that describe the context of the CI job.

For example, it includes details such as groupid, grouppath, projectid, and projectpath.

On the GCP side, Workload Identity Federation allows the use of OIDC to grant GCP IAM roles to the external identities represented by the ID tokens. Through Workload Identity Federation, the GCP user can grant specific IAM roles to specific principals identified through the OIDC claim. For example, a particular groupid claim can be given an IAM role to access a particular set of secrets in GCP Secrets Manager. This would allow the GCP user to grant granular access to the secrets in GCP Secrets Manager.

CI configuration on GitLab Rails

GitLab Rails will be the interface where users configure the CI jobs. For the GCP Secrets Manager integration, there needs to be additional configuration to specify GCP Secrets Manager as a source for external secrets as well as GCP specific information in order to enable authentication between GitLab Runner and GCP Secrets Manager.

The proposed CI keyword would be the following:

```
yaml
jobname:
  idtokens:
    GCPSMIDTOKEN:
      aud: my-GCP-workload-identity-federation-audience
  secrets:
    DATABASEPASSWORD:
      gcpsm:
        name: my-project-secret This is the name of the secret defined in GCP Secrets Manager
        version: 1 optional: default to latest.
```

token: GCPSMIDTOKEN

In addition, GitLab Runner needs to know the following in order to perform the authentication and access the secret.

These should be included as CI variables in the job.

- GCP Project Number GCPPROJECTNUMBER
- GCP Workload Federation Pool ID GCPWORKLOADFEDERATIONPOOLID
- GCP Workload Federation Provider ID GCPWORKLOADFEDERATIONPROVIDERID

Secrets access by GitLab Runner

Based on the job specification defined above, GitLab Runner needs to implement the following:

1. OIDC authentication with GCP Secure Token Service to obtain an access token.
1. Secret access requests to GCP Secrets Manager to obtain the payload of the desired secret version.
1. Adding the secrets to the build.

Alternative Solutions

N/A.

SECTION: engineering/architecture/design-documents/ci_job_token/_index.md

<!-- vale gitlab.FutureTense = NO -->
{{< engineering/design-document-header >}}

Summary

GitLab CI is a Continuous Integration platform widely used to run various jobs, builds, and pipelines.

Each CI job is provided with a CI job token (a kind of a security token) that allows it to interact with other GitLab APIs to accomplish a task. Currently, this token has the same level of access as the user who triggered the pipeline, which violates the principle of least privilege (PoLP).

This proposal outlines the development needed to reduce the access granted by this token in order to adhere to the principle of least privilege, while delivering incremental value along the way.

Motivation

Currently, when a CI job runs, it is provided with a CIJOBTOKEN, which the job uses to interact with GitLab resources. This token is tied to the identity of the user who triggered the CI job, carrying a fixed set of permissions based

on that user's roles and access levels.

When an API request includes a CIJOBTOKEN, the authorization process checks the role of the user who triggered the pipeline and restricts access to the API using CI allowlist rules. However, this method of computing the runtime access of the CIJOBTOKEN can unintentionally expose new APIs, increasing the risk of abuse.

We do support more types of tokens than just a CI job token. Authorization behavior varies based on the type of token presented, and these differences can create gaps in enforcement as new API endpoints are introduced or modified.

This proposal introduces a different kind of a CIJOBTOKEN, following a standardized format we might be able to apply across the various token types in use today, simplifying the process of authorization.

Goals

This proposal aims to establish a consistent format for generating a CI job token, allowing for more granular control over access scopes. In particular, it seeks to limit the scope of the CIJOBTOKEN by defining the minimum necessary permissions for each token.

- The CIJOBTOKEN should be ephemeral and grant only the minimal required access.
- Permissions CI job tokens can have should be customizable per project.
- The token should support extensions, such as the inclusion of fields like organizationid.
- Current CIJOBTOKEN behavior should be preserved following our breaking changes policy.

Non-Goals

- A Security Token Service will not be created.
- Reducing the duration of CIJOBTOKEN access is not a focus.
- Unifying PAT scopes with custom abilities will not be pursued.
- Token types will not be consolidated into a single token.
- The permission set for the CIJOBTOKEN will not be expanded.
- We will not handle cases where specific projects can override group-level permissions.

Proposal

This document proposes a change in the handling of CIJOBTOKEN to enhance security and fine-tune resource access. Projects will gain the ability to specify which resources CI job tokens can access. Access can be configured by project or by group.

The token's final set of permissions will be the result of the intersection of the following three models:

- Job Token Scope (allowlist): Serves as a boundary, specifying the maximum allowable access the token can grant. (This is configured by Maintainer+)

- Fine-grained Permissions: Specifies the allowed resources a token can access. (This is configured by Maintainer+)
- User Role: Defines the overall permissions of the user, ensuring proper auditability and supporting different environments or refs, such as protected versus non-protected branches

To determine the exact level of access granted to a token, access will be progressively narrowed:

1. First, by limiting access to projects that appear on the allowlist.
1. Then, by restricting access based on the permissions defined for each project on the allowlist.
1. Finally, the user role permissions will limit token access.

Proposed Design

Since fine-grained permissions are an extension of allowlists, we can only apply fine-grained permissions on endpoints that enforce allowlists. Allowlists are enforced on endpoints that use the routesettings :authentication metadata:

```
ruby
routesetting :authentication, jobtokenallowed: true
```

Fine-grained permission will be using the routesettings :authorization metadata, for example:

```
ruby
routesetting :authorization, jobtokenpolicies: :readpackages,
  allowpublicaccessforenabledprojectfeatures: :packageregistry
```

If this metadata is defined on an endpoint, it is saying the Packages resource should have at least Read permissions defined in the allowlist entry.

The allowpublicaccessforenabledprojectfeatures keyword says fine-grained permissions do not need to be enforced if the accessed project is public or internal and the specified project features have their visibility set to 'Everyone with Access'.

Fine-grained permissions will be enforced on all endpoints that enforce allowlists. The ci:jobtokens:checkpolicies rake task, that is run as part of static analysis on pipelines and when pushing changes, ensures this. The same script also ensures the defined permissions are valid.

Example

Let's say usera has developer access to both projecta and projectb and projectb is a private

project with
both allowlists and fine-grained permissions enabled.

Now usera can trigger a pipeline from projecta that accesses an endpoint on projectb by authenticating with a CIJOBTOKEN. For example projecta/.gitlab-ci.yml might look like this:

```
yaml
build-job:
  script:
    - 'curl -s https://gitlab.com/api/v4/projects/{projectb.id}/environments -H "JOB-TOKEN:
$CIJOBTOKEN"'
```

This specific endpoint is defined (in lib/api/environments.rb) like this:

```
ruby
routesetting :authentication, jobtokenallowed: true
routesetting :authorization, jobtokenpolicies: :readenvironments,
  allowpublicaccessforenabledprojectfeatures: [:repository, :builds, :environments]
get ':id/environments' do
  ...
```

This means that when usera triggers a pipeline in projecta, this job will fail unless the allowlist entry for projecta, defined in projectb, has the Environments resource set to either Read or Read and write permissions.

Flowchart

This is a flowchart that shows the decisions made to determine the accesibility of an endpoint.

mermaid

flowchart TD

```
Start[A user triggers a job that accesses an API endpoint for a project] --> A
A -- Public or internal --> B
A -- Private --> C
B -- Everyone With Access --> B1[Access granted]
B -- Only Project Members --> C
C -- true --> D
C -- false --> C1[Access denied]
D -- true --> E
D -- false --> D1[Access granted]
E -- true --> H[Fine-grained permissions enabled?]
H -- true --> F
H -- false --> H1[Access granted]
E -- false --> E1[Access denied]
F -- true --> F1[Access granted]
F -- false --> F2[Access denied]
```


A{Project accessibility}
B{Project feature visibility}
C{User is a project member with the right role?}
D{Allowlists enabled?}
E{Project allowlisted?}
F{Permissions authorized?}

Permissions

The complete list of endpoints that can currently be controlled by fine-grained permissions can be seen here: <https://docs.gitlab.com/ci/jobs/finegrainedpermissions/available-api-endpoints>. This list is automatically generated by running:

```
shell
bundle exec rake ci:jobtokens:compiledocs
```

This list is ensured to be up-to-date by the `ci:jobtokens:checkdocs` rake task, which is included as part of the `ci:jobtokens:checkpolicies` rake task, that is run in the `ci-job-token-policies-verify` job in the pipeline and when pushing changes from lefthook.

Alternative Solutions

- Build a Security Token Service
 - Pros: Standards compliant solution
 - Cons: Additional up front effort and maintenance is required before we can realize any value.
- Migrating to the GitLab OAuth2 provider
 - Pros: Standards compliant solution
 - Cons: More effort is required up front before any value can be realized
- Do nothing
 - Pros: No work required
 - Cons: See the Motivation section of this document for details

SECTION: engineering/architecture/design-documents/ci_pipelines_for_github/_index.md

{{< engineering/design-document-header >}}

Summary

Some customers using GitHub Source Code Management want to integrate with GitLab's CI solutions to run pipelines and leverage our security features. They do not want to mirror the source code on GitLab and want instantaneous pipelines for their pushes.

Motivation

Our current approach with GitHub <-> GitLab integration is with mirroring at a minimum 5 minute interval. This is too slow for feedback, uses long lived personal access tokens, and requires a copy of GitHub's source code on GitLab. Additionally, as the entire project and all pipelines are run under the user that initiates the integration; this could lead to a permission mismatch.

With an effective solution, we can sell GitLab CI/CD and other Ops features to businesses that use GitHub as a source control management tool.

CI tools in the market currently use GitHub source to run CI/CD externally. Buildkite, CircleCi, TeamCity, Jenkins are examples where the runner pulls directly from GitHub. In such examples, the pipeline config file can live either on GitHub or in the services.

Goals

As an initial MVC we want to support

1. A GitHub App (on/off GitHub's marketplace - this doesn't need to be in MVP, users can install with a direct link)
 1. Near instant pipeline creation upon GitHub pushes
 1. We will only be communicating with GitHub via short lived tokens
 1. Correct user management system, through direct user mapping
 1. Each user on GitHub's side should have a billable seat on GitLab
 1. GitLab users should have the least privilege needed to run pipelines
 1. Runners are the only place to interact (fetch/pull) with the source code
 1. Customer source code is stored in a GitHub Repo and is never stored in a GitLab repo

Non-Goals

Items that are out of scope (For MVP) include in order of importance

1. GitLab Self-Managed
 1. Why: This should be quite do-able. Assuming we have docs for SM users to set-up. We should get this from the MVP
1. GitHub Enterprise
 1. Why: This will require extra set-up, licensing, testing, and manual configurations as GitHubApp and Actions are different on the Enterprise level
1. GitLab Security Scans, Pipeline execution policies, Merge Request widgets from artifacts (JUnit/Cobertura)
 1. Why: Because the source code is in GitHub and our scans only work natively there'll be extra work to fetch and analyze the code
 1. Similar for the MR widget, since the Merge Request is on GitHub side we can't modify it
1. Different actions on GitHub side that might want/require CI such as
 1. Forks
 2. Tags
 3. Branch name changes, etc
 4. Why: These are bit more complex as they rely on different webhooks as well as sha configurations

These are do-able, but just to reduce scope and complexity we can iterate on additional

features.

Proposal

GitHub will communicate with GitLab via our GitHubApp via webhooks.

GitLab will generate a user access token via the GitHub App for the user that triggered the webhook, and create a pipeline for that user.

When runners poll GitLab's api, GitLab will provide the GitHub repo location and access token for the runner to fetch the code from.

GitLab will then use GitHub's API to update the commit with the pipeline status.

Design and implementation details

Prerequisites

The process begins with a GitHub organization owner creating the projects in GitLab that would be used as CI/CD.

We would reuse CI/CD For External Repositories, and have a "checkbox" to not mirror the repository (UI pending). During the import we'll save the GitHub repository id. Issue for discussion

After import, we would automatically enable a new GitLab integration called GitHub SCM for these projects.

Then the customer will use a direct link to install our GitHub App, and choose which repos on GitHub to install it on. Upon GitHub app installation, and for OAuth, each user will need to accept the terms and conditions listed in . The App's permissions will be set-up by GitLab and hosted on GitHub. This will be set up with the basic permission of (readrepository, and writecommitstatus). We can modify these permissions and add new ones anytime via Modifying GitHub App, although this will require users to re-accept the conditions and re-authenticate.

Each user that would want to trigger pipeline would need to OAuth with GitHub. Either via OAuth login or connecting their GitLab account with GitHub

Once everything is set-up properly

The steps here will be in accordance with the diagram above

1. User on GitHub initiates a push to a branch
1. GitHub will automatically trigger a webhook via our GitHubApp. This'll send a push payload to GitLab
 1. GitLab Webhook Push Payload
1. GitLab will use the signed payload's senderid and map that to a GitLab user.
 1. If the GitLab user does not have permissions to run pipelines. A pipeline will be created but will fail immediately. Anyone with correct permissions can re-try this pipeline.
 1. If the GitLab user does have permissions to create pipelines.
 1. GitLab will use the refreshtoken of the user to generate a new accesstoken, so we can

act on behalf of the user

1. This accesstoken is repository specific, and will only have
 1. Read access to repos
 1. Write access to commitstatus (to update the commit with pipeline details)
1. There's no way to revoke the accesstoken without also revoking the refreshtoken. But if want to be safe, we can just re-generate a new accesstoken after the pipeline has finished.
1. The pipeline will be generated by pulling a .gitlab-ci.yml file present on GitHub side.
 1. To make this easier in the future to work with includes; we'll require this .yml file to be in a gitlab folder
1. GitLab calls CreatePipelineService sidekiq job and sets up the pipeline.
1. Rails will pass the necessary params including the accesstoken to the runner. It will also update the pipeline on GitHub side to "running" via GitHub API.
1. The runner will directly call GitHub with the accesstoken to fetch and pull the repository for that branch
1. When the pipeline finishes, runner will update GitLab as normal
1. GitLab will use the installation access token to post a commit on GitHub with the pipeline's sha. Notifying them the commit has a finished pipeline.
 1. GitHub commit status API
 1. This will be limited to the 5k/15k rate limits per hour. But this should be sufficient for our customers.

Step 3 expanded:

There are many ways this authorization could play out. More details in Corresponding Implementation Issue.

We will enforce a 1:1 user mapping on GitHub <-> GitLab. Currently this is product's preferred approach. This allows us to make sure the product is properly licensed with the correct number of seats.

Each user will need to manually link their GitHub accounts to their GitLab profiles. Which can only be done once GitHub is used as an authentication provider.

The downside is that users or bot accounts that do not have a GitLab account mapping will not be able to run pipelines. The workaround is to create a failed pipeline that maintainers can manually run.

Step 4.4 - As the gitlab-ci.yml file is hosted externally on GitHub.com. Only this file will then be pulled in when a webhook is triggered. This'll keep the source code and config file in sync.

There are certain yaml definitions which would not work with this approach. The includes: yaml definition would not work, similarly cross-pipeline configs, security scans, pipeline test reports, execution policies would need additional patching to work.

Alternative Solutions

For Step 3.

Another idea is to use service accounts. Have the customer create a service account they want

to use and assign it to a project with the permission set or custom role. Then we'll have a UI allowing them to select which service account they want to use for these GitHub webhook actions.

This'll need composite identities, as cross-project pipelines can exist. Additionally, there'll be extra considerations for billing as GitHub users can share GitLab seats

For Step 4.

Another option is for the GitHub App to use Installation Access Tokens.

This would be a great option if not for the rate limits. As IAT have a limit of 5k/hour calls for GitHub.com and 15k/hour for GitHub Enterprise, this is not enough for large customers without a caching mechanism in place. This limit is on the organization level and shared between all their repositories.

Authentication as a GitHub App installation

Questions for Reviewers

1. Compliance Question: Do we need additional terms and conditions depending on how we run & keep GitHub code?

1. As the job logs will be kept on GitLab side

1. Answer: (Rutshah) should be okay to keep job logs

2. Compliance Question/PM: Some customers don't want anything else aside from runner interacting with the source code. This requirement is unclear; does referencing, have snippets of file/artifact names in the job logs invalidate this requirement?

1. Partial Answer: (Rutshah) These passby references should be okay

SECTION: engineering/architecture/design-documents/ci_pipeline_components/_index.md

{{< engineering/design-document-header >}}

Summary

The goal of the CI/CD pipeline components catalog is to make the reusing pipeline configurations easier and more efficient. Providing a way to discover, understand and learn how to reuse pipeline constructs allows for a more streamlined experience. Having a CI/CD pipeline components catalog also sets a framework for users to collaborate on pipeline constructs so that they can be evolved and improved over time.

This design doc used to define the architectural guidelines on how to build a CI/CD catalog of pipeline components. Since we've done it, you can find a link to the official user documentation below along with the historic content of the document.

For more information on the feature, see the CI/CD Components documentation.

The archived version of the blueprint file can be found [here](#).

Who

Architectural Design:

<!-- vale gitlab.Spelling = NO -->

Role	Who
-----	-----
Author	Fabio Pitino
Engineering Leaders	Cheryl Li, Mark Nuzzo
Product Manager	Dov Hershkovitch
Architecture Evolution Coaches	Kamil Trzeci-ski, Grzegorz Bizon

DRIs:

Role	Who
-----	-----
Product	Dov Hershkovitch
Engineering	Avielle Wolfe, Laura Montemayor
UX	Sunjung Park

<!-- vale gitlab.Spelling = YES -->

SECTION: engineering/architecture/design-documents/ci_pipeline_processing/_index.md

{{< engineering/design-document-header >}}

Summary

GitLab CI is one of the oldest and most complex features in GitLab.

Over the years its YAML syntax has considerably grown in size and complexity.

In order to keep the syntax highly stable over the years, we have primarily been making additive changes

on top of the existing design and patterns.

Our user base has grown exponentially over the past years. With that, the need to support their use cases and customization of the workflows.

While delivering huge value over the years, the various additive changes to the syntax have also caused

some surprising behaviors in the pipeline processing logic.

Some keywords accumulated a number of responsibilities, and some ambiguous overlaps were discovered among

keywords and subtle differences in behavior were introduced over time.

The current implementation and YAML syntax also make it challenging to implement new features.

In this design document, we will outline a streamlined approach to improve pipeline behavior

predictability

and reduce the configuration burden on users, ultimately strengthening GitLab's product competitiveness.

Goals

Business Goals

- Enhance Product Competitiveness: By reducing configuration complexity and improving pipeline predictability,

GitLab will offer a more intuitive and robust CI/CD experience. This positions GitLab as the preferred choice

for both new and existing users, helping to attract and retain customers, including those with highly complex workflows.

- Increase Development Efficiency: Clarifying keyword responsibilities and simplifying the pipeline model

reduces code complexity, which improves maintainability and decreases the time and resources needed for future enhancements.

The development teams will have greater agility to implement new features and address issues quickly.

Product Goals

- Provide a clear, consistent pipeline configuration model that reduces ambiguity and allows users to more accurately control pipeline behavior.

- Create a cohesive, predictable model for DAG and STAGE configurations, enabling users to seamlessly integrate both without risk of unexpected behavior.

- Simplify GitLab CI's codebase to make future improvements more manageable and reduce the maintenance burden on GitLab's engineering team.

- Facilitate a migration path for existing customers without introducing any breaking changes.

Problem Statement

- Ambiguity and Overlapping Keyword Roles: Some keywords, like when and allowfailure, have multiple roles that overlap,

leading to unpredictable behavior. Users find it difficult to anticipate outcomes, especially in complex pipelines.

This ambiguity increases support cases and frustrates users, who may seek alternative solutions.

- Related Issues: 233876, 382179,

Epic,

17759, 17397.

- Inconsistent Pipeline Models: The STAGE and DAG models do not always behave consistently, making it challenging for users to configure pipelines that use both models without unintended side effects.

This inconsistency adds a learning curve and reduces GitLab's appeal for complex pipeline needs.

- Related Issues: 233712, 219371,

388866, 20237.

Non-Goals

We will not discuss how to avoid breaking changes for now.

Motivation

The list of problems is the main motivation for this design document.
Most of these problems have been discussed before in the "Restructure CI job when keyword" epic.

Problem 1: The responsibility of the when keyword

Right now, the when keyword has many responsibilities;

- > - onsuccess (default): Run the job only when no jobs in earlier stages fail or have allowfailure: true.
- > - onfailure: Run the job only when at least one job in an earlier stage fails. A job in an earlier stage
 - > with allowfailure: true is always considered successful.
- > - never: Don't run the job regardless of the status of jobs in earlier stages.
- > Can only be used in a rules section or workflow: rules.
- > - always: Run the job regardless of the status of jobs in earlier stages. Can also be used in workflow:rules.
- > - manual: Run the job only when triggered manually.
- > - delayed: Delay the execution of a job
- > for a specified duration.

It answers three questions;

- What's required to run? => onsuccess, onfailure, always
- How to run? => manual, delayed
- Add to the pipeline? => never

As a result, for example; we cannot create a manual job with when: onfailure.
This can be useful when persona wants to create a job that is only available on failure, but needs to be manually played.
For example; publishing failures to dedicated page or dedicated external service.

Problem 2: Abuse of the allowfailure keyword

We control the blocker behavior of a manual job by the allowfailure keyword.
Actually, it has other responsibilities; "determine whether a pipeline should continue running when a job fails".

Currently, a manual job;

- is not a blocker when it has allowfailure: true (by default)
- a blocker when it has allowfailure: false.

As a result, for example; we cannot create a manual job that is allowfailure: false and not a

blocker.

yaml

job1:

stage: test

when: manual

allowfailure: true default

script: exit 0

job2:

stage: deploy

script: exit 0

Currently;

- job1 is skipped.
- job2 runs because job1 is ignored since it has allowfailure: true.
- When we run/play job1;
 - if it fails, it's marked as "success with warning".

allowfailure with rules

allowfailure becomes more confusing when using rules.

From docs:

- > The default behavior of allowfailure changes to true with when: manual.
- > However, if you use when: manual with rules, allowfailure defaults to false.

From docs:

- > The default value for allowfailure is:
 - >
 - > - true for manual jobs.
 - > - false for jobs that use when: manual inside rules.
 - > - false in all other cases.

For example;

yaml

job1:

stage: build

script: ls

when: manual

nextjob1:

stage: test

```
script: exit 0
```

```
job2:
```

```
  stage: test
```

```
  script: ls
```

```
  rules:
```

```
    - if: $ALWAYSTRUE != "asdsad"
```

```
      when: manual
```

```
nextjob2:
```

```
  stage: deploy
```

```
  script: exit 0
```

job1 and job2 behave differently;

- job1 is not a blocker because it has allowfailure: true by default.
- job2 is a blocker rules: when: manual does not return allowfailure: true by default.

Problem 3: Different behaviors in DAG/needs

The main behavioral difference between DAG and STAGE is about the "skipped" and "ignored" states.

Background information:

- skipped:
 - When a job is when: onsuccess and its previous status is failed, it's skipped.
 - When a job is when: onfailure and its previous status is not "failed", it's skipped.
- ignored:
 - When a job is when: manual with allowfailure: true, it's ignored.

Problem:

The skipped and ignored states are considered successful in the STAGE processing but not in the DAG processing.

Problem 3.1. Handling of ignored status with manual jobs

Example 1:

```
yaml
```

```
build:
```

```
  stage: build
```

```
  script: exit 0
```

```
  when: manual
```

```
  allowfailure: true by default
```

```
test:
  stage: test
  script: exit 0
  needs: [build]
```

- build is ignored (skipped) because it's when: manual with allowfailure: true.
- test is skipped because "ignored" is not a successful state in the DAG processing.

Example 2:

```
yaml
build:
  stage: build
  script: exit 0
  when: manual
  allowfailure: true by default
```

```
test:
  stage: test
  script: exit 0
```

- build is ignored (skipped) because it's when: manual with allowfailure: true.
- test2 runs and succeeds.

Problem 3.2. Handling of skipped status with when: onfailure

Example 1:

```
yaml
buildjob:
  stage: build
  script: exit 1
```

```
testjob:
  stage: test
  script: exit 0
```

```
rollbackjob:
  stage: deploy
  needs: [buildjob, testjob]
  script: exit 0
  when: onfailure
```

- buildjob runs and fails.
- testjob is skipped.
- Even though rollbackjob is when: onfailure and there is a failed job, it is skipped because the needs list has a "skipped" job.

Example 2:

```
yaml
buildjob:
  stage: build
  script: exit 1

testjob:
  stage: test
  script: exit 0

rollbackjob:
  stage: deploy
  script: exit 0
  when: onfailure
```

- buildjob runs and fails.
- testjob is skipped.
- rollbackjob runs because there is a failed job before.

Problem 4: The skipped and ignored states

Let's assume that we solved the problem 3 and the "skipped" and "ignored" states are not different in DAG and STAGE.

How should they behave in general? Are they successful or not? Should "skipped" and "ignored" be different?

Let's examine some examples;

Example 4.1. The ignored status with manual jobs

```
yaml
build:
  stage: build
  script: exit 0
  when: manual
  allowfailure: true by default

test:
  stage: test
  script: exit 0
```

- build is in the "manual" state but considered as "skipped" (ignored) for the pipeline processing.
- test runs because "skipped" is a successful state.

Alternatively;

yaml

build1:

```
  stage: build
  script: exit 0
  when: manual
  allowfailure: true by default
```

build2:

```
  stage: build
  script: exit 0
```

test:

```
  stage: test
  script: exit 0
```

- build1 is in the "manual" state but considered as "skipped" (ignored) for the pipeline processing.
- build2 runs and succeeds.
- test runs because "success" + "skipped" is a successful state.

Example 4.2. The skipped status with when: onfailure

yaml

build:

```
  stage: build
  script: exit 0
  when: onfailure
```

test:

```
  stage: test
  script: exit 0
```

- build is skipped because it's when: onfailure and its previous status is not "failed".
- test runs because "skipped" is a successful state.

Alternatively;

```
yaml
build1:
  stage: build
  script: exit 0
  when: onfailure

build2:
  stage: build
  script: exit 0

test:
  stage: test
  script: exit 0
```

- build1 is skipped because it's when: onfailure and its previous status is not "failed".
- build2 runs and succeeds.
- test runs because "success" + "skipped" is a successful state.

Problem 5: The dependencies keyword

The dependencies keyword is used to define a list of jobs to fetch artifacts from. It is a shared responsibility with the needs keyword. Moreover, they can be used together in the same job. We may not need to discuss all possible scenarios but this example is enough to show the confusion;

```
yaml
test2:
  script: exit 0
  dependencies: [test1]
  needs:
    - job: test1
    artifacts: false
```

Information 1: Canceled jobs

Are a canceled job and a failed job the same? They have many differences so we could easily say "no".

However, they have one similarity; they can be "allowed to fail".

Let's define their differences first;

- A canceled job;
- It is not a finished job.

- Canceled is a user requested interruption of the job. The intent is to abort the job or stop pipeline processing as soon as possible.
- We don't know the result, there is no artifacts, etc.
- Its eventual state is "canceled" so no job can run after it.
 - There is no when: oncanceled.
 - Even when: always is not run.
- A failed job;
 - It is a machine response of the CI system to executing the job content. It indicates that execution failed for some reason.
 - It is equal answer of the system to success. The fact that something is failed is relative, and might be desired outcome of CI execution, like in when executing tests that some are failing.
 - We know the result and there can be artifacts.
 - Its eventual state is "failed" so subsequent jobs can run depending on their when values.
 - when: onfailure and when: always are run.

The one similarity is; they can be "allowed to fail".

```
yaml
build:
  stage: build
  script:
    - sleep 10
    - exit 1
  allowfailure: true
```

```
test:
  stage: test
  script: exit 0
  when: onsuccess default
```

If build runs and gets canceled, then test runs.

If build runs and gets failed, then test runs.

An idea on using canceled instead of failed for some cases

There is another aspect. We often drop jobs with a failurereason before they get executed, for example when the namespace ran out of compute minutes or when limits are exceeded. Dropping jobs in the failed state has been handy because we could communicate to the user the failurereason for better feedback. When canceling jobs for various reasons we don't have a way to indicate that.

We cancel jobs because the user ran out of Compute Credits while the pipeline was running, or because the pipeline is auto-canceled by another pipeline or other reasons.

If we had a stopreason instead of failurereason we could use that for both cancelled and failed jobs
and we could also use the canceled status more appropriately.

Information 2: Empty state

We recently updated the documentation of the when keyword for clarification;

- > - onsuccess: Run the job only when no jobs in earlier stages fail or have allowfailure: true.
- > - onfailure: Run the job only when at least one job in an earlier stage fails.

For example;

```
yaml
test1:
  when: onsuccess
  script: exit 0
  needs: [] would lead to the same result
```

```
test2:
  when: onfailure
  script: exit 0
  needs: [] would lead to the same result
```

- test1 runs because there is no job failed in the previous stages.
- test2 does not run because there is no job failed in the previous stages.

The onsuccess means that "nothing failed", it does not mean that everything succeeded. The same goes to onfailure, it does not mean that everything failed, but does mean that "something failed".

This semantic goes by a expectation that your pipeline succeeds, and this is happy path. Not that your pipeline fails, because then it requires user intervention to fix it.

Technical expectations

All proposals or future decisions must follow these goals;

1. The allowfailure keyword must only responsible for marking failed jobs as "success with warning".
 - Why: It should not have another responsibility, such as determining a manual job is a blocker or not.
 - How: Another keyword will be introduced to control the blocker behavior of a manual job.
1. With allowfailure, canceled jobs must not be marked as "success with warning".
 - Why: "canceled" is a different state than "failed".
 - How: Canceled with allowfailure: true jobs will not be marked as "success with warning".
1. The when keyword must only answer the question "What's required to run?". And it must be the

only source of truth

for deciding if a job should run or not.

1. The when keyword must not control if a job is added to the pipeline or not.
 - Why: It is not its responsibility.
 - How: Another keyword will be introduced to control if a job is added to the pipeline or not.
1. The "skipped" and "ignored" states must be reconsidered.
 - TODO: We need to discuss this more.
1. A new keyword structure must be introduced to specify if a job is an "automatic", "manual", or "delayed" job.
 - Why: It is not the responsibility of the when keyword.
 - How: A new keyword will be introduced to control the behavior of a job.
1. The needs keyword must only control the order of the jobs. It must not be used to control the behavior of the jobs
 - or to decide if a job should run or not. The DAG and STAGE behaviors must be the same.
 - Why: It leads to different behaviors and confuses users.
 - How: The needs keyword will only define previous jobs, like stage does.
1. The needs and dependencies keywords must not be used together in the same job.
 - Why: It is confusing.
 - How: The needs and dependencies keywords will be mutually exclusive.

Proposal

N/A

Design and implementation details

This will be determined after the proposal is approved.

Breaking changes, implementation details, and migration paths will be discussed in this phase.

Feedback

Please share your feedback at the feedback issue.

SECTION: engineering/architecture/design-documents/ci_scale/_index.md

{{< engineering/design-document-header >}}

Summary

GitLab CI/CD is one of the most data and compute intensive components of GitLab.

Since its initial release in 2012,

the CI/CD subsystem has evolved significantly. It was integrated into GitLab in September 2015 and has become one of the most beloved CI/CD solutions.

GitLab CI/CD has come a long way since the initial release, but the design of the data storage for pipeline builds remains almost the same since 2012. We store all the builds in PostgreSQL in cibuilds table, and because we are

creating more than 5 million builds each day on GitLab.com we are reaching database limits that are slowing our development velocity down.

On February 1st, 2021, GitLab.com surpassed 1 billion CI/CD builds created. In February 2022 we reached 2 billion of CI/CD build stored in the database. The number of builds continues to grow exponentially.

The screenshot below shows our forecast created at the beginning of 2021, that turned out to be quite accurate.

Goals

Enable future growth by making processing 20M builds in a day possible.

Challenges

The current state of CI/CD product architecture needs to be updated if we want to sustain future growth.

We were running out of the capacity to store primary keys: DONE

The primary key in cibuilds table is an integer value, generated in a sequence. Historically, Rails used to use integer type when creating primary keys for a table. We did use the default when we created the cibuilds table in 2012. The behavior of Rails has changed since the release of Rails 5. The framework is now using bigint type that is 8 bytes long, however we have not migrated primary keys for cibuilds table to bigint yet.

In early 2021 we had estimated that would run out of the capacity of the integer type to store primary keys in cibuilds table before December 2021. If it had happened without a viable workaround or an emergency plan, GitLab.com would go down. cibuilds was just one of many tables that were running out of the primary keys available in Int4 sequence.

Before October 2021, our Database team had managed to migrate all the risky tables' primary keys to big integers.

See the related Epic for more details.

Some CI/CD database tables are too large: IN PROGRESS

There is more than two billion rows in cibuilds table. We store many terabytes of data in that table, and the total size of indexes is measured in terabytes as well.

This amount of data contributes to a significant number of performance

problems we experience on our CI PostgreSQL database.

Most of the problems are related to how PostgreSQL database works internally, and how it is making use of resources on a node the database runs on. We are at the limits of vertical scaling of the CI primary database nodes and we frequently see a negative impact of the cibuilds table on the overall performance, stability, scalability and predictability of the CI database GitLab.com depends on.

The size of the table also hinders development velocity because queries that seem fine in the development environment may not work on GitLab.com. The difference in the dataset size between the environments makes it difficult to predict the performance of even the most simple queries.

Team members and the wider community members are struggling to contribute the Verify area, because we restricted the possibility of extending cibuilds even further. Our static analysis tools prevent adding more columns to this table. Adding new queries is unpredictable because of the size of the dataset and the amount of queries executed using the table. This significantly hinders the development velocity and contributes to incidents on the production environment.

We also expect a significant, exponential growth in the upcoming years.

One of the forecasts done using Facebook's Prophet shows that in the first half of 2024 we expect seeing 20M builds created on GitLab.com each day. In comparison to around 5M we see created today. This is 10x growth from numbers we saw in 2021.

Status: As of October 2021 we reduced the growth rate of cibuilds table by writing build options and variables to cibuildsmetadata table. We are also working on partitioning the largest CI/CD database tables using time decay pattern.

Queuing mechanisms were using the large table: DONE

Because of how large the table is, mechanisms that we used to build queues of pending builds (there is more than one queue), were not very efficient. Pending builds represented a small fraction of what we store in the cibuilds table, yet we needed to find them in this big dataset to determine an order in which we wanted to process them.

This mechanism was very inefficient, and it had been causing problems on the production environment frequently. This usually resulted in a significant drop of the CI/CD Apdex score, and sometimes even caused a significant performance degradation in the production environment.

There were multiple other strategies that we considered to improve performance and

reliability. We evaluated using Redis queuing, or a separate table that would accelerate SQL queries used to build queues. We decided to proceed with the latter.

In October 2021 we finished shipping the new architecture of builds queuing on GitLab.com.

We then made the new architecture generally available.

Moving big amounts of data is challenging: IN PROGRESS

We store a significant amount of data in cibuilds table. Some of the columns in that table store a serialized user-provided data. Column cibuilds.options stores more than 600 gigabytes of data, and cibuilds.yamlvariables more than 300 gigabytes (as of February 2021).

It is a lot of data that needs to be reliably moved to a different place. Unfortunately, right now, our background migrations are not reliable enough to migrate this amount of data at scale. We need to build mechanisms that will give us confidence in moving this data between columns, tables, partitions or database shards.

Effort to improve background migrations will be owned by our Database Team.

Status: In progress. We plan to ship further improvements that will be described in a separate architectural blueprint.

Proposal

Below you can find the original proposal made in early 2021 about how we want to move forward with CI Scaling effort:

- > Making GitLab CI/CD product ready for the scale we expect to see in the upcoming years is a multi-phase effort.
- >
- > First, we want to focus on things that are urgently needed right now. We need to fix primary keys overflow risk and unblock other teams that are working on database partitioning and sharding.
- >
- > We want to improve known bottlenecks, like builds queuing mechanisms that is using the large table, and other things that are holding other teams back.
- >
- > Extending CI/CD metrics is important to get a better sense of how the system performs and to what growth should we expect. This will make it easier for us to identify bottlenecks and perform more advanced capacity planning.
- >
- > Next step is to better understand how we can leverage strong time-decay characteristic of CI/CD data. This might help us to partition CI/CD dataset to reduce the size of CI/CD database tables.

Iterations

Work required to achieve our next CI/CD scaling target is tracked in the CI/CD Scaling epic.

- 1. · Migrate primary keys to big integers on GitLab.com.
- 1. · Implement the new architecture of builds queuing on GitLab.com.
- 1. · Make the new builds queuing architecture generally available.
- 1. Partition CI/CD data using time-decay pattern.

Status

Created at 21.01.2021, approved at 26.04.2021.

Status: In progress.

Who

Proposal:

<!-- vale gitlab.Spelling = NO -->

Role	Who
-----	-----
Author	Grzegorz Bizon
Architecture Evolution Coach	Kamil Trzci-ski
Engineering Leader	Cheryl Li
Product Manager	Jackie Porter
Domain Expert / Verify	Fabio Pitino
Domain Expert / Database	Jose Finotto
Domain Expert / PostgreSQL	Nikolay Samokhvalov

DRIs:

Role	Who
-----	-----
Leadership	Cheryl Li
Product	Jackie Porter
Engineering	Grzegorz Bizon

Domain experts:

Area	Who
-----	-----
Domain Expert / Verify	Fabio Pitino
Domain Expert / Verify	Marius Bobin
Domain Expert / Database	Jose Finotto
Domain Expert / PostgreSQL	Nikolay Samokhvalov

<!-- vale gitlab.Spelling = YES -->

SECTION: engineering/architecture/design-documents/clickhouse_ingestion_pipeline/_index.md

{{< engineering/design-document-header >}}

Table of Contents

- Summary
 - Why
 - How
- Motivation
- Case Studies
 - Replicating existing data into ClickHouse
 - Ingesting large volumes of data into ClickHouse
- Goals
- Non-goals
- General considerations
- Challenges building this
- Proposed solution
- Design & Implementation
- References

Summary

Develop a scalable & reliable data ingestion abstraction to help efficiently ingest large volumes of data from high throughput systems into ClickHouse.

Why

To enable any application at GitLab to write necessary data into ClickHouse regardless of the scale at which they generate data today, or in the future. Refer to Motivation for why ClickHouse in the first place.

How

By building a write abstraction (API/Library) that allows a user to write data into ClickHouse and has all necessary configurations, conventions and best-practices around instrumentation, service-discovery, etc, built into it out of the box.

Motivation

ClickHouse is an online, analytical processing (OLAP) database that powers use-cases that require fetching real-time, aggregated data that does not mutate a lot. ClickHouse is highly performant and can scale to large volumes of data as compared to traditional transactional relational databases (OLTP) such as Postgres, MySQL. For further reading around ClickHouse's capabilities, see [1], [2] and [3].

At GitLab, our current and future ClickHouse uses/capabilities reference & describe multiple use-cases that could be facilitated by using ClickHouse as a backing datastore. A majority of

these talk about the following two major areas of concern:

1. Being able to leverage ClickHouse's OLAP capabilities enabling underlying systems to perform an aggregated analysis of data, both over short and long periods of time.

1. The fact that executing these operations with our currently existing datasets primarily in Postgres, is starting to become challenging and non-performant.

Looking forward, assuming a larger volume of data being produced by our application(s) and the rate at which it gets produced, the ability to ingest it into a more capable system, both effectively and efficiently helps us scale our applications and prepare for business growth.

Case studies

From an initial assessment of all (reported) use-cases that intend to utilise ClickHouse, the following broad patterns of usage can be observed:

1. Efficiently replicating existing data from other databases into ClickHouse, most prominently Postgres.

1. Directly ingesting large volumes of data into ClickHouse for asynchronous processing, data aggregation & analysis.

The following section(s) explain details of each problem-domain:

1. Replicating existing data into ClickHouse

With due reference to our prior work around this, it has been established that logical replication from Postgres is too slow. Instead, we'll need to be able to emit data change events within database transactions which can then get processed asynchronously to write or update corresponding data in ClickHouse.

The following case-studies describe how these groups intend to solve the underlying problem:

- ~group::optimize has been working towards a scalable PostgreSQL data replication strategy which can be implemented on the application layer.

- Proposal: Scalable data sync/replication strategy talks about such a strategy and the additional challenges with using Sidekiq for queueing/batching needs.

- It has been observed that pumping data from PostgreSQL into ClickHouse directly might not be the right way to approach the problem at hand.

- In addition to the problems described above, another class of problems when replicating data across systems is also the handling of data backfill and/or data migrations that happen upstream.

- group::data has been working around syncing data from some of our Postgres databases into a Snowflake-based data warehouse. See this issue for options considered: List down all possible options for postgres to snowflake pipeline before designing the current system in place.

- With the work done around our Next Gen GitLab SaaS Data Pipeline, the data team owns a

"custom" pipeline that does incremental data extractions based on an updatedat timestamp column. This helps import a significant subset of operational database relations into Snowflake data-warehouse.

- As the volume of data grows, we can foresee this (ETL) pipeline warranting more time and resources to execute resulting in delays across the time between data being produced and being available in Snowflake data-warehouse.

- We might also see data inconsistency/incompleteness issues emanating from the current setup since row deletions are not transferred into Snowflake, inflating data volume and skewing analysis. Any information about multiple updates happening between import interval period are also lost.

- Having a scalable ingestion pipeline that can help replicate data from our databases into an intermediate system and/or ClickHouse in near real-time would help improve the operational characteristics around this system.

2. Ingesting large volumes of data into ClickHouse

We need to be able to ingest large volumes of potentially unaggregated data into ClickHouse which may result into a large number of small writes as well. This can have an adverse effect on how ClickHouse processes and stores incoming data. To mitigate this problem, we need to queue & batch smaller writes into larger ones to keep the ingestion pipeline efficient at all times.

The following case-studies describe how each group intends to solve the underlying problem:

- ~group::observability explains their need of ingesting large amounts of data into ClickHouse, with the following two issues:

- Proposal: GitLab Observability Platform - Data Ingestion talks about using an external events store, such as Kafka, to first ingest data as received from users, then writing it into ClickHouse in larger batches thereby eliminating the need to write a large number of small writes without hampering write performance from how ClickHouse MergeTree processes ingested data.

- In addition, ClickHouse: Investigate client-side buffering to batch writes into ClickHouse talks about their experimentation with using application-local queueing/batching to work around the problems mentioned above.

- ~"group::analytics instrumentation" has been working on building our analytics offering and recently looking at building and/or improving parts of the system.

- Product Analytics Collector Component talks about replacing Jitsu with Snowplow for collecting and processing tracking events. For more details of the proposal, see Jitsu replacement.

- The initial design was prototyped with Snowplow as Jitsu Replacement PoC.

- From the design, it is easy to observe how large amounts of data will be ingested into

ClickHouse and could potentially benefit from the use of a scalable ingestion pipeline.

Goals

Well-defined, established client abstractions

We want to define and establish a fully-functional application-side abstraction that can help ingest data into ClickHouse without getting in the way of how an application itself is designed while keeping the underlying code backend-agnostic. The proposed abstraction should become the default choice for any applications, core or satellite, at GitLab.

Support for high throughput in volume of writes

A solution here should enable an application to write any amount of inserts (order of upto 1000-5000 writes per second) to the underlying database efficiently while also allowing for growth as the application scales out. Considering how ClickHouse processes incoming writes, a proposed solution should be able to batch a number of very small writes into larger batches.

Reliable, consistent delivery of data

A solution here should also ensure reliable & consistent delivery of ingested data into the underlying database minimising undue loss of data before being eventually persisted into ClickHouse.

Non-goals

Addressing data types, schemas or formats

At this stage of this proposal, we're not optimizing for addressing which data types, schemas or formats we receive ingested data in. It should be delegated to the backend-specific implementations themselves and not handled within the write abstraction.

Addressing where our data sources exist today

We're also not addressing any client-side specific details into the design at this point. The write abstraction should only remain a tool for the language in which it is written. As long as an application can use it to write data as any other third-party library, we should be good to build on top of it.

General Considerations

Having addressed the details of the two aforementioned problem-domains, we can model a proposed solution with the following logical structure:

- Ingestion
 - APIs/SDKs
 - HTTP2/gRPC Sidecar
- Transport & Routing
 - Multi-destination
- Digestion/Compute

- Enrichment
- Processing
- Persisting

Major challenges around building such a capability

Self-managed environments

The single, biggest challenge around introducing ClickHouse and related systems would be the ability to make it available to our users running GitLab in self-managed environments. The intended goals of this proposal are intentionally kept within those constraints. It is also prudent to establish that what we're proposing here be applicable to applications consuming ClickHouse from inside self-managed environments.

There are ongoing efforts to streamline distribution and deployment of ClickHouse instances for managed environment within the larger scope of ClickHouse Usage at GitLab. A few other issues tackling parts of the aforementioned problem are:

- Research and understand component costs and maintenance requirements of running a ClickHouse instance with GitLab
- ClickHouse maintenance and cost research

Wide variety of data sources, their structures & usage patterns

The data that we intend to ingest into ClickHouse can come from a wide variety of data sources and be structured in different schemas or formats. With that considered, it's non-trivial effort to draft a solution that suffices all use-cases efficiently.

Should we decide to build an intermediate ingestion system, any solution should help provide a source/schema/format-agnostic data transport layer with an established, matured client-abstraction to maximise the number of applications that can use it.

Building on top of our current database infrastructure

Our current database infrastructure operates at a fairly large scale and adding more applications that continuously read/write against it adds to the pressure on the existing resources. It's important we move away any workloads and/or datasets that can be safely processed in a different context altogether.

Service Discovery

We're still normalising the details around distribution and deployment of ClickHouse clusters and/or instances for our applications. Subject to how we end up doing it, for a client to be able to discover which ClickHouse cluster, shard or table would need to become a part any such solution.

Proposed Solution

In light of the problems discussed earlier, it'd be in our better interests to allow the usage of an external, intermediate system subject to what one's needs might be especially around the

volume & scale of data being written from an application into ClickHouse.

Therefore, we intend to develop an abstraction that can enable an application to store data into ClickHouse regardless of the scale that they (currently) operate at. It also:

- Facilitates an application to switch from one technology to another should their performance and/or scale requirements change over time.
- Allows for backend-specific conventions, configurations & best practices such as instrumentation, service-discovery, etc. to be encoded in one place for all applications to leverage consistently.

Design & Implementation

Core assumptions

- We're only going to focus on writing data into ClickHouse as mentioned in aforementioned non-goals. With details of how our data lands into ClickHouse, this document does not (intentionally) address where this data comes from. Some of those details are delegated to the applications generating this data i.e as long as they can consume this abstraction, they should be able to write data into ClickHouse.
- We're going to delegate the choice of different storage backends to a following blueprint or epic since that's outside the scope of this design. With ClickHouse as the eventual destination for our data, this document only talks about writing data into it - either directly or indirectly via a queueing/batching system.

Architecture

Having an abstraction around writing data help client-side instrumentation to stay backend-agnostic allowing them to switch code paths depending on where it runs.

An example setup should look like:

ruby

```
Gitlab::Database::Writer.config do |config|
```

```
  when using sync mode, data gets written directly into ClickHouse,  
  therefore, it's also assumed the backend here is ClickHouse  
  config.mode = :sync OR :async  
  config.backend = :clickhouse optional  
  OR
```

```
  when using async mode, data is written to an intermediate system  
  first, then written into ClickHouse asynchronously  
  config.mode = :async  
  config.backend = :pubsub OR :kafka OR :otherbackend
```

```
  then backend-specific configurations hereafter
```

```
config.url = 'tcp://user:pwd@localhost:9000/database'
```

```
  for example, a serializer helps define how data travels over the wire
```

```
config.jsonserializer = ClickHouse::Serializer::JsonSerializer
```

```
...
```

```
end
```

```
G
```